Ahorros y Depósitos

Alcance funcional y alcance de la automatización — Grupo 5

Curso de Automatización de Pruebas de Software · Universidad Nacional de Asunción — CIT

Profesor: Steven Gracia

Integrantes

Subgrupo 05: Jessica Escobar

Otros integrantes: _____

Noviembre de 2026 · Repositorio: github.com/stevenayal/curso_automatizacion

Alcance funcional

Fuente: requerimientos RF-G5-01 a RF-G5-10

Ahorro programado

Plan con una **meta** y un **aporte mensual**. El saldo crece con cada aporte debitado de una cuenta y el plan se **completa** al alcanzar la meta.

RF-G5-01 a 06: listar, crear, consultar, modificar, dar de baja y aportar.

Depósito a plazo

Un **importe inmovilizado** desde una cuenta por un plazo y una tasa pactados, con su interés proyectado. Puede **cancelarse antes** cobrando el interés proporcional a los días transcurridos.

RF-G5-07 a 10: listar, constituir, consultar y cancelar.

Fuera de alcance del sandbox

- Débito automático mensual de los aportes (cada aporte se dispara a mano) y retiro o rescate del ahorro.
- Pago de intereses sobre el ahorro (su tasa es informativa), capitalización (todo el interés es simple) e impuestos.
- Vencimiento automático de un depósito y su renovación; penalidades por cancelación anticipada.

Actores: **cliente del banco** (titular de cuentas y productos), **API REST** (/api/v2) y **aplicación web** (/v2) que consume la misma API a través de un proxy.

Reglas de negocio clave

Qué se verifica con las pruebas

- **Interés simple, base 365:** monto × tasa / 100 × plazo / 365. Ej.: 5.000.000 al 11,5 % a 180 días proyecta **283.561,64**; 2.000.000 al 10 % a 90 días proyecta **49.315,07**.
- **Rangos:** plazo entero de 30 a 1095 días, tasa de 0 a 999,99 % (0 es válida) y monto mayor que cero.
- **Constituir:** la cuenta debe existir y estar activa con saldo suficiente; depositar exactamente el saldo es válido. El importe sale de la cuenta y el depósito nace **activo**. La operación es atómica.
- **Cancelar anticipadamente:** solo un depósito activo; acredita capital más interés prorrateado por los días transcurridos (tope: el plazo), sin penalidad. Cancelado el mismo día, el interés es 0. Si no está activo: 409.
- **Inmutabilidad:** un depósito no se modifica ni se da de baja (PUT y DELETE responden 405).
- **Estados:** activo, vencido y cancelado. La API **no vence** depósitos automáticamente: «vencido» solo existe en datos sembrados.
- **Ahorro:** cada aporte debita la cuenta y suma al plan; al alcanzar la meta queda completado y ya no admite aportes ni cambios.

La pantalla es más estricta que la API

Reglas de UI frente a reglas de la API

Regla	Pantalla (aplicación web)	API REST
Monto mínimo del depósito	PYG 1.000.000: rechaza en el navegador	Acepta cualquier monto positivo (sin mínimo de negocio)
Plazo	Lista fija: 30, 60, 90, 180, 365, 730 y 1095 días	Cualquier entero entre 30 y 1095 (29 y 1096 dan 400)
Tasa	Sugerida por plazo: 3,5 % a 30 días hasta 8 % a 1095 días; editable	No tiene tarifario: acepta cualquier tasa hasta 999,99
Saldo insuficiente	Rechaza en el navegador antes de llamar a la API	409 CONFLICT: «Saldo insuficiente para constituir el depósito.»
Cuentas ofrecidas	Solo cuentas activas en guaraníes del titular	Cualquier cuenta existente, incluso de otro cliente
Cancelación	Botón en el detalle con confirmación del interés estimado	POST /depositos/{id}/cancelar; 409 si no está activo

Por eso cada escenario se etiqueta @**web** o @**api**: la misma regla de negocio puede cumplirse en una capa y no en la otra.

Alcance de la automatización

Cuatro capas, un mismo módulo

BDD + UI

Gherkin en español con **Cucumber** y **Playwright**. Page Object en playwright/pages, steps y hooks separados. Tags @grupo-05, @web, @api, @smoke.

API

Colecciones **Postman** ejecutadas con **Newman**. Patrón SQL REST: el pre-request obtiene datos y el post-request verifica en la base.

Rendimiento

Planes **JMeter** sobre GET y POST /depositos con CSV y correlación de ids, y dashboard de **Grafana** como evidencia.

CI/CD

GitHub Actions: cada capa corre en un workflow, genera su **informe PDF** con Python y sube artifacts.

Principios de diseño

- Datos propios y aislados: los escenarios crean lo que usan (fixture) en vez de depender de filas sembradas del sandbox compartido.
- Los escenarios @web de solo lectura llevan una **guarda de red** que aborta cualquier POST, PUT, PATCH o DELETE; los @fixture escriben únicamente sobre datos que ellos mismos crean.
- Secretos solo desde **secrets.API_KEY**; el runner de 05-01 y el workflow del PR #65 enmascaran la key en los resultados antes de publicar artifacts.

BDD: de los requerimientos a escenarios

Cucumber + Playwright

24

escenarios en el feature actual

14 @web · 10 @api · 0 @wip en main

- **Punto de partida:** 11 escenarios genéricos, todos @wip.
- **Primera entrega:** 3 ejecutables por pantalla; luego 12 ejecutables @web (ejecución local: **12 de 12 aprobados**).
- **Fixture aislado:** el equipo docente sumó escenarios @api y flujos de UI que crean sus propios datos (@fixture).

Contradicciones entre el escenario original y el comportamiento real

- «Depósito que **vence hoy**»: la API no vence depósitos; hay activos con días restantes negativos. Se reescribió como «un depósito vencido sembrado se muestra como Vencido, con su interés generado y sin cancelación».
- «Intereses **generados**» de un depósito vigente: la pantalla muestra interés **proyectado**; el generado existe solo tras cancelar o vencer.
- «Importe menor al **mínimo**» y «plazo **no permitido**»: son reglas de la pantalla; la API acepta ambos casos dentro de sus rangos.
- Valores esperados **solo del documento** (283.561,64; 49.315,07; códigos HTTP 400, 404, 405 y 409).

Estado en CI: en la corrida de main del 29/10, 3 de 28 filas de Ahorros y depósitos pasaron y 25 fallaron; el log registra «Falta API_KEY» en el fixture (necesita la key del Curso 2). La causa de los demás fallos está por confirmar.

API: Postman + Newman

Colección del Grupo 5 (subgrupo 05)

15 escenarios: depósito vigente y vencido, constituir sin saldo, cancelar, plan de ahorro por id, depósito inexistente (404), plazo no permitido (29 y 1096 días), total del saldo, monto mínimo, tasas, intereses, aportar y eliminar un plan. Cada request obtiene o crea sus datos con SQL en el pre-request. Workflow en revisión (PR #65).

Colección del subgrupo 05-01

9 escenarios más flujos E2E con validación SQL: constituir (INSERT), plazo inválido, cancelar (UPDATE) y cancelar dos veces (409).

Última corrida exitosa (1/11): 21 requests, **47 aserciones, 0 fallos**, 29 s. Informe PDF y JUnit publicados como artifact.

Patrón pre y post request

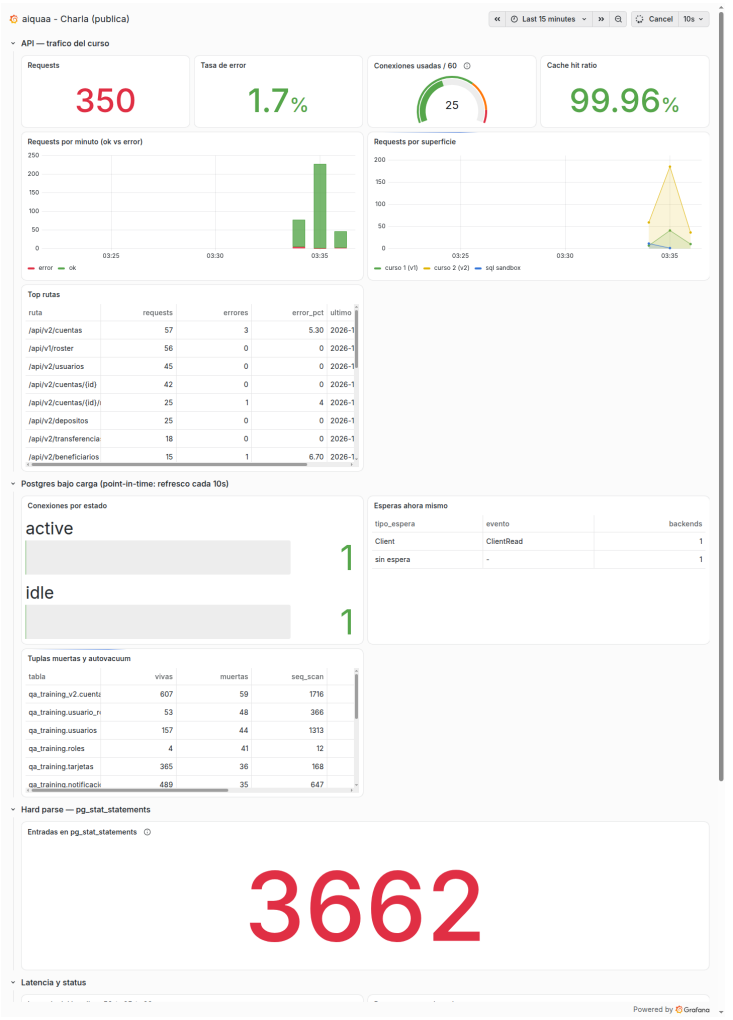
- Pre-request: SELECT parametrizado para elegir un depósito activo, vencido o un plan existente; nunca se interpolan valores en el SQL.
- Post-request: se confirma en la base lo que hizo el endpoint (estado, saldo y movimiento de la cuenta).
- Límite del sandbox: 30 requests por minuto por credencial; los workflows usan --delay-request y manejan el 429.

Rendimiento: JMeter + Grafana

Request	Muestras	Errores	Media (ms)	Máx (ms)
GET listar por usuario y estado	5	1 (20 %)	158	194
GET consultar con id correlacionado	5	1 (20 %)	159	183
POST crear depósito desde CSV	6	1 (16,7 %)	325	1064
POST cancelar depósito (limpieza)	5	2 (40 %)	172	194
Total	**21**	**5 (23,81 %)**	**209**	**1064**

Lectura honesta del resultado: el workflow terminó en verde, pero 5 de 21 muestras fallaron (p95 ≈ 979 ms). El pipeline publica el informe sin bloquear por el porcentaje de error; queda como hallazgo a revisar.

- Plan del subgrupo 05: **P_GRUPO5_AHORROS_DEPOSITOS.jmx** (GET y POST /depositos, 60 requests por defecto) aún **no ejecutado** en CI.
- Umbrales del flujo modelo de la clase (echo): error rate máximo 1 % y p95 máximo 800 ms; este plan no los bloquea.



Captura del dashboard de Grafana (artifact)

Pipelines en GitHub Actions

Estado verificado en el repositorio

Workflow	Qué hace	Último resultado observado
grupo-05-01-ahorros-depositos.yml	Newman + informe PDF (05-01)	1/11 verde: 21 requests, 47 de 47 aserciones
Y_GRUPO_05_01_DEPOSITOS_jmeter.yml	JMeter + Grafana + PDF (05-01)	1/11 verde, con 23,81 % de muestras con error
bdd-pom-ci.yml	Cucumber + Playwright + PDF de evidencias	Workflow verde el 1/11, pero el reporte lista 19 filas fallidas de Ahorros (05 y 05-01). En main (29/10): 3 aprobadas y 25 fallidas
qa-checks.yml	Validaciones de calidad	Verde el 1/11; rojo en el PR #65 (sin secret en PR desde fork)
grupo-05-ahorros-depositos.yml	Newman + PDF del subgrupo 05	PR #65 abierto: check rojo «Falta el secret API_KEY»
Y_GRUPO5_jmeter.yml	JMeter + Grafana + PDF (flujo echo)	Incorporado; sin corrida en el repositorio del docente

La API key se lee siempre de **secrets.API_KEY**. Algunos pasos auxiliares (captura de Grafana, evidencias) usan continue-on-error, por eso un workflow verde no implica que todos los escenarios hayan pasado. El workflow del PR #65 falla de forma explícita si falta el secret y no suprime códigos de salida.

Reportes y evidencias

Qué genera cada pipeline

Artifact	Contenido	Origen
grupo-05-01-ahorros-depositos-reportes	Informe PDF de Newman, JUnit y JSON de resultados	Newman + reporter de Python
grupo-05-01-jmeter-grafana-reportes	Informe PDF de rendimiento, dashboard HTML, JTL y captura de Grafana	JMeter + Selenium + reporter de Python
semana-7-bdd-pom-evidencias	Reporte Cucumber (JSON y HTML) y PDF de evidencias con capturas del sandbox	Cucumber + Playwright
grupo-05-ahorros-depositos-newman-reportes	Informe PDF y resultados de Newman con la API key enmascarada	PR #65 (pendiente de merge)

Los artifacts se conservan 14 días. Los informes PDF incluyen portada, resumen de aserciones, detalle por request y enlace a la ejecución.

Evidencia local del BDD: reports/cucumber-report.html (12 escenarios ejecutables aprobados en la versión previa a la ampliación del docente).

Hallazgos de calidad del módulo

Lo que las pruebas dejan en evidencia

- **Sin control de titularidad:** la API no verifica que la cuenta indicada pertenezca al titular; cualquier credencial opera sobre cualquier cliente.
- **Sin monto mínimo en la API:** la pantalla exige PYG 1.000.000 pero la API acepta montos menores (existen depósitos de PYG 1 en el sandbox).
- **Tarifario solo en la pantalla:** la API acepta cualquier tasa hasta 999,99 %.
- **Ahorro sin rescate:** el dinero aportado no se puede retirar; DELETE da de baja un plan con saldo y ese saldo queda inaccesible.
- **Vencimiento no automático:** hay depósitos activos con fecha vencida y días restantes negativos.
- **Sandbox compartido:** otras suites modifican los datos sembrados (el depósito sembrado 1 ya figura cancelado), por eso se usan fixtures propios.
- **Límite de 30 requests por minuto:** exige demoras y reintentos en los pipelines.

Conclusiones y próximos pasos

Estado real al cierre

Logrado

• Requerimientos traducidos a escenarios BDD con valores esperados del documento.
• Cuatro capas automatizadas: UI, API, rendimiento y CI/CD con informes PDF.
• Contradicciones entre escenarios y comportamiento real detectadas y documentadas.
• Newman 05-01: 47 de 47 aserciones en CI.

Pendiente

• Merge del PR #65 y primera corrida del Newman del subgrupo 05 con la API key.
• Ejecutar el plan JMeter P_GRUPO5_AHORROS_DEPOSITOS en CI.
• Revisar el 23,81 % de errores de la corrida de rendimiento y definir un umbral que bloquee.
• Que el job de BDD reciba API_KEY para los escenarios @fixture.

Mensaje final: la automatización no solo valida lo que el sistema hace, también muestra dónde el sistema real se aparta de lo que el negocio espera. Esas diferencias son el principal resultado del trabajo.

Enlaces

Para revisar el trabajo

- Repositorio: **github.com/stevenayal/curso_automatizacion**
- Requerimientos: docs/requerimientos/grupo-05-ahorros-depositos.pdf
- Feature BDD: grupos/grupo-05-ahorros-depositos/features/ahorros-depositos.feature
- Colecciones Postman: grupos/grupo-05-ahorros-depositos/postman y grupos/grupo-05-01-ahorros-depositos/postman
- Planes JMeter: tests/performance/plans (P_GRUPO5_AHORROS_DEPOSITOS.jmx y P_GRUPO_05_01_DEPOSITOS.jmx)
- Workflows: .github/workflows (grupo-05-01-ahorros-depositos, Y_GRUPO_05_01_DEPOSITOS_jmeter, bdd-pom-ci, qa-checks)
- Pull request del workflow Newman del subgrupo 05: **github.com/stevenayal/curso_automatizacion/pull/65**